LOVELY PROFESSIONAL UNIVERSITY

PROJECT REPORT

AI Resume Analyzer

AI-powered resume-to-job description analysis platform

INT333 - DevOps Project

Submitted by	[Student Name]
Registration number	[Registration Number]
Roll number / section	[Roll Number] / [Section]
Submitted to	[Faculty / Project Guide]
Department	Computer Science and Engineering
Institution	Lovely Professional University
Submission period	October 2026

Abstract

This report presents Careerly AI Resume Analyzer, a Flask web application that compares a PDF resume with a job description and returns a structured, AI-generated review. The system extracts PDF text in memory, removes common direct identifiers, requests consent before sending text to Groq, and presents an ATS-style score, matching and missing skills, strengths, weaknesses, and suggestions. A job-description optimizer can structure supplied text or create a clearly labeled general draft from a short role title. The project also demonstrates Docker packaging, GitHub Actions testing and image publication, a Render deployment blueprint, and supporting operational examples for Ansible, Terraform, Puppet, and Nagios. The report documents implementation, security considerations, testing, deployment, limitations, and future work.

Keywords: Resume analysis, Flask, Groq, Docker, GitHub Actions, CI/CD, Render, DevOps, privacy

Table of Contents

Abstract

1. Introduction

2. Requirements and Technology Stack

3. System Architecture and Workflow

4. Application Implementation

5. DevOps, CI/CD, and Deployment

6. Security and Privacy

7. Testing and Verification

8. Installation and Operation

9. Future Enhancements

10. Conclusion

References

1. Introduction

Project overview

Careerly AI Resume Analyzer is a web application that compares a resume with a target job description. It extracts text from a PDF resume and asks a hosted large language model to return a structured review for the applicant.

Problem statement

Applicants often have limited feedback about how their resumes align with a particular vacancy. Manually comparing skills and tailoring a resume can be time-consuming. The project provides a repeatable, role-specific review to help users identify alignment and possible gaps.

Objectives

The objectives are to accept PDF resumes securely, compare extracted resume text with job requirements, present an ATS-style score and skill comparison, provide actionable feedback, and demonstrate containerization, CI/CD, cloud deployment configuration, and basic operations tooling.

Scope

The system is an advisory resume-review application. It does not submit applications, modify a user resume, guarantee a real ATS score, or replace a recruiter. Resume files are not retained by the application.

2. Requirements and Technology Stack

Functional requirements

The user can upload a PDF (maximum 16 MB), enter or optimize job-description text, explicitly consent to sending text to the AI provider, and review the returned score, matching skills, skills to develop, strengths, weaknesses, and suggestions.

Software stack

Backend: Python and Flask. Frontend: server-rendered HTML, CSS, and vanilla JavaScript. PDF text extraction: PyPDF2. AI inference: Groq API using openai/gpt-oss-120b by default. Configuration: python-dotenv. Delivery: Docker, GitHub Actions, and Render.

Runtime versions

The CI test job runs on Python 3.11. The Docker image uses python:3.13-slim. The README documents Python 3.11 or newer for local setup.

Operational prerequisites

A Groq API key is required for analysis and optimization. Docker Hub username and access-token secrets are required for image publishing from pushes to main. Render requires GROQ_API_KEY as a service environment variable.

3. System Architecture and Workflow

Architecture

The browser communicates with a Flask application. Flask validates the request, extracts PDF text in memory, redacts common direct identifiers, and sends the remaining resume and job-description text to Groq after user consent. Groq returns structured analysis, which Flask serves as JSON and the browser renders.

Request flow

1) Select a PDF and enter a job description or role title. 2) Confirm the AI-processing disclosure. 3) Flask validates the upload and extracts text. 4) The application redacts common email, phone, street-address, and link patterns. 5) The Groq chat-completions API evaluates the prompt. 6) The browser displays the response.

Application endpoints

GET / serves the user interface. GET /health returns process status and whether an AI key is configured. POST /optimize structures detailed job text or creates a clearly labeled general starter draft when input looks like a short role title. POST /analyze accepts the PDF, job description, and consent and returns the structured review.

Analysis output

The report contains an ATS-style score (0-100), matching skills, missing or underrepresented skills, strengths, weaknesses, suggestions, and an overall summary. Results are presented together, with the skills comparison first so applicants can see the match and gaps immediately.

4. Application Implementation

Resume processing

The Flask request class uses an in-memory stream for uploaded files. The server checks that a file is present, has a PDF extension, is within the configured request-size limit, and contains extractable text. Scanned-image PDFs without a text layer are not OCR-processed and return an explanatory error.

AI integration

The application calls Groq chat completions with a configurable GROQ_MODEL. The current default is openai/gpt-oss-120b. Resume analysis requests JSON with a defined output shape; invalid JSON is logged and returned as an application error rather than silently accepted.

Job-description optimization

Detailed descriptions are reorganized into Role Summary, Required Skills, Nice-to-Have Skills, Responsibilities, and Qualifications while the prompt instructs the model to preserve supplied requirements. Short, single-line role-title input instead requests a general starter draft and the interface labels it as AI-generated for review. Users should tailor such a draft to the actual employer posting.

User interface

The responsive interface includes PDF upload, file replacement/removal, job-description editing, consent, optimization, progress and error states, and a report. Results show the matched and missing-skill groups first, followed by score and summary, strengths and improvement areas, and suggested next steps.

5. DevOps, CI/CD, and Deployment

Docker

The Dockerfile starts from python:3.13-slim, installs Python dependencies, copies the application, binds the Flask process to 0.0.0.0 in the container, and runs app.py. The Docker image does not embed the Groq API key; secrets are provided at runtime.

GitHub Actions

The workflow runs unit tests for pushes to main and pull requests targeting main. Pull requests also build a Docker image without publishing it. A main-branch push requires DOCKER_USERNAME and DOCKER_TOKEN, logs in to Docker Hub, then publishes latest and a commit-SHA tag to abhijeetchy51/ai-resume-analyser. Missing publishing secrets fail the job explicitly.

Render deployment

render.yaml defines a Docker web service built from the repository Dockerfile, declares /health as the health-check endpoint, and requires GROQ_API_KEY to be supplied outside source control. The Render Blueprint builds from the repository configuration; the workflow separately publishes images to Docker Hub.

Configuration management and monitoring examples

Ansible provides a localhost playbook that checks the public Render /health endpoint and requires SERVICE_URL. Terraform contains a safe provider/version starter but no Render infrastructure resources. Puppet demonstrates package installation for a separately managed Linux monitoring host. Nagios provides an HTTP health-check command and example host/service configuration. These examples do not administer Render infrastructure or its private container host.

6. Security and Privacy

Data handling

The application processes the uploaded PDF in memory and does not save it to the project uploads directory. Before AI processing, common email addresses, phone numbers, street addresses, and links are redacted. This is partial redaction, not full anonymization; names and other personal details may remain.

Consent and external processing

The interface explains that remaining resume text and job-description text are sent to Groq and requires explicit consent before analysis or optimization. Users should not submit information they are not comfortable sharing with the provider and should review the provider?s data-handling terms.

Application safeguards

API keys are configured through environment variables, .env files are excluded from Git, responses use no-store cache headers, request size is limited, CORS is restricted to local development origins, and Flask debug mode is disabled when the application starts directly.

Limitations

Pattern-based redaction can miss identifiers or remove non-identifying strings. The AI output may be incomplete or incorrect. The ATS-style score is an estimate, not a validated score from a commercial applicant-tracking system. The application does not currently provide account management, persistent history, OCR, or human review.

7. Testing and Verification

Automated test suite

The repository uses Python unittest. The current suite contains five tests: health endpoint response and no-store header; all report sections rendered without tabs and skills listed first; the Groq client receives the configured default model; a short role title follows the labeled-draft prompt path; and a detailed description retains the no-invented-requirements instruction.

Validation performed

The test command `python -m unittest discover -s tests -v` completed successfully with 5 tests passing during report preparation. The frontend inline JavaScript was also checked with Node.js syntax validation during the latest UI changes.

Manual verification checklist

For a live demonstration, verify that the application opens, /health returns status ok, a text-based PDF can be analyzed after consent, skill matches and gaps appear first, optimization handles both a full description and a title-only draft, and a missing/invalid file produces a clear error. CI and Render deployment state can change after this report is prepared; check the current Actions and Render dashboards before claiming a live run.

8. Installation and Operation

Local installation

Clone https://github.com/abhijeetkumar51/ai-resume-analyzer, enter the repository directory, install requirements with `python -m pip install -r requirements.txt`, create a private .env file next to app.py with GROQ_API_KEY, then run `python app.py`. Open http://localhost:5000.

Docker operation

Build with `docker build -t ai-resume-analyzer .` and run with `docker run --rm -p 5000:5000 -e GROQ_API_KEY=your_key_here ai-resume-analyzer`. Do not put a real key in a committed file or command history; for regular use, pass secrets through an appropriate local secret mechanism.

Required configuration

GROQ_API_KEY is required for AI features. GROQ_MODEL is optional and defaults to openai/gpt-oss-120b. PORT is read from the environment, defaulting to 5000. HOST defaults to 127.0.0.1 for local execution; the Dockerfile sets it to 0.0.0.0.

9. Future Enhancements

Planned improvements

Potential future work includes OCR for scanned resumes, richer PDF validation, user-controlled redaction previews, configurable model/provider selection with availability checks, response schema validation, rate limiting, optional user accounts with explicit retention controls, accessibility testing, and expanded integration tests.

Operational improvements

Further work could add dependency/image scanning, pinned GitHub Action revisions, Docker build caching and provenance, deployment smoke tests, monitoring/alerts, and a documented recovery process. Any persistence or analytics feature should be introduced only with a clear privacy policy and user consent.

10. Conclusion

Conclusion

The project demonstrates an end-to-end AI web application and a practical DevOps workflow: input validation, PDF text extraction, privacy-aware redaction, a hosted model integration, a responsive interface, Docker packaging, automated tests, CI/CD publication, and a Render deployment blueprint. It also distinguishes real deployment responsibilities from starter examples for Ansible, Terraform, Puppet, and Nagios. The result is a useful prototype for resume-to-role comparison, with clear limitations around AI reliability, redaction coverage, and actual ATS scoring.

References

- Project repository: https://github.com/abhijeetkumar51/ai-resume-analyzer

- Groq model documentation: https://console.groq.com/docs/models

- Flask documentation: https://flask.palletsprojects.com/

- Docker documentation: https://docs.docker.com/

- GitHub Actions documentation: https://docs.github.com/actions

- Render documentation: https://render.com/docs